Python PANDAS Questions

1. Create a Pandas Series from a list of numbers.

```
data = [10, 20, 30, 40]
s = pd.Series(data)
s
```

✓ 0.0s

```
0    10
1    20
2    30
3    40
dtype: int64
```

2. Create a DataFrame from a dictionary of lists.

```
data = {
    "Name": ["A", "B", "C"],
    "Age": [21, 22, 23]
}
df = pd.DataFrame(data)
df
```

✓ 0.0s

	Name	Age
0	A	21
1	B	22
2	C	23

3. Read a CSV file into a DataFrame.

```
df = pd.read_csv("student_performance_updated_1000.csv")
df
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
0	1.0	John	Male	85.0	15.0	78.0	1.0	High	80.0	4.8	59.0	False
1	2.0	Sarah	Female	90.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True
2	3.0	Alex	Male	78.0	10.0	65.0	0.0	Low	68.0	4.6	92.0	False
3	4.0	Michael	Male	92.0	25.0	90.0	3.0	High	92.0	2.9	96.0	False
4	5.0	Emma	Female	NaN	18.0	82.0	2.0	Medium	85.0	4.1	97.0	True
...	...	...	...	...	...	...	...	...	...	...	...	...
995	NaN	Kenneth Murray	Male	85.0	20.0	NaN	1.0	High	72.0	0.8	80.0	True
996	4497.0	Amy Stout	Female	91.0	NaN	86.0	0.0	High	90.0	3.9	80.0	True
997	1886.0	NaN	Male	85.0	8.0	82.0	2.0	Low	68.0	0.4	54.0	False
998	7636.0	Joseph Sherman	Male	88.0	17.0	60.0	2.0	High	85.0	0.9	53.0	True
999	8021.0	Maria Walls	Female	88.0	10.0	90.0	1.0	Medium	NaN	2.4	94.0	True

000 rows × 12 columns

4. Display the first 10 rows of a DataFrame.

```
df.head(10)
```
✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
0	1.0	John	Male	85.0	15.0	78.0	1.0	High	80.0	4.8	59.0	False
1	2.0	Sarah	Female	90.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True
2	3.0	Alex	Male	78.0	10.0	65.0	0.0	Low	68.0	4.6	92.0	False
3	4.0	Michael	Male	92.0	25.0	90.0	3.0	High	92.0	2.9	96.0	False
4	5.0	Emma	Female	NaN	18.0	82.0	2.0	Medium	85.0	4.1	97.0	True
5	6.0	Olivia	Female	95.0	30.0	88.0	1.0	High	NaN	2.8	97.0	False
6	7.0	Daniel	Male	70.0	8.0	60.0	0.0	Low	62.0	4.5	96.0	False
7	8.0	Sophia	Female	NaN	17.0	77.0	1.0	Medium	78.0	1.0	70.0	True
8	9.0	James	Male	82.0	12.0	70.0	2.0	Low	72.0	3.6	50.0	False
9	10.0	Isabella	Female	91.0	22.0	86.0	3.0	High	88.0	2.9	59.0	True

5. Display the last 5 rows of a DataFrame.

```
df.tail(5)
```
✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
995	NaN	Kenneth Murray	Male	85.0	20.0	NaN	1.0	High	72.0	0.8	80.0	True
996	4497.0	Amy Stout	Female	91.0	NaN	86.0	0.0	High	90.0	3.9	80.0	True
997	1886.0	NaN	Male	85.0	8.0	82.0	2.0	Low	68.0	0.4	54.0	False
998	7636.0	Joseph Sherman	Male	88.0	17.0	60.0	2.0	High	85.0	0.9	53.0	True
999	8021.0	Maria Walls	Female	88.0	10.0	90.0	1.0	Medium	NaN	2.4	94.0	True

6. Get the shape (rows, columns) of a DataFrame.

```
df.shape
```
✓ 0.0s

```
(1000, 12)
```

7. List all column names of a DataFrame.

```
df.columns
```
✓ 0.0s

```
Index(['StudentID', 'Name', 'Gender', 'AttendanceRate', 'StudyHoursPerWeek',
       'PreviousGrade', 'ExtracurricularActivities', 'ParentalSupport',
       'FinalGrade', 'Study Hours', 'Attendance (%)', 'Online Classes Taken'],
      dtype='str')
```

8. Select a single column from a DataFrame.

```
df["ExtracurricularActivities"]
```

✓ 0.0s

```
0      1.0
1      2.0
2      0.0
3      3.0
4      2.0
      ...
995    1.0
996    0.0
997    2.0
998    2.0
999    1.0
Name: ExtracurricularActivities, Length: 1000, dtype: float64
```

9. Select multiple columns from a DataFrame.

```
df[['Name', 'Gender', 'AttendanceRate']]
```

✓ 0.0s

	Name	Gender	AttendanceRate
0	John	Male	85.0
1	Sarah	Female	90.0
2	Alex	Male	78.0
3	Michael	Male	92.0
4	Emma	Female	NaN
...	...	...	...
995	Kenneth Murray	Male	85.0
996	Amy Stout	Female	91.0
997	NaN	Male	85.0
998	Joseph Sherman	Male	88.0
999	Maria Walls	Female	88.0

000 rows × 3 columns

10. Filter rows where a column value is greater than 90.

```
df[df["AttendanceRate"] > 90]
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
3	4.0	Michael	Male	92	25.0	90.0	3.0	High	92.0	2.9	96.0	False
5	6.0	Olivia	Female	95	30.0	88.0	1.0	High	NaN	2.8	97.0	False
9	10.0	Isabella	Female	91	22.0	86.0	3.0	High	88.0	2.9	59.0	True
12	4611.0	Katherine Gray	Female	91	18.0	78.0	0.0	Low	62.0	1.7	59.0	True
16	NaN	Courtney Clark	NaN	95	25.0	82.0	2.0	Low	68.0	2.7	65.0	True
...	...	...	...	...	...	...	...	...	...	...	...	...
982	5410.0	Christina Johnson	Male	91	22.0	78.0	3.0	NaN	80.0	0.2	55.0	False
987	7350.0	Samantha Klein	Male	92	22.0	86.0	2.0	High	88.0	3.1	81.0	True
989	2116.0	Kimberly Pena	Female	91	30.0	88.0	2.0	High	68.0	3.6	79.0	False
992	NaN	James Garcia	Male	91	15.0	90.0	2.0	High	68.0	4.3	68.0	False
996	4497.0	Amy Stout	Female	91	NaN	86.0	0.0	High	90.0	3.9	80.0	True

297 rows × 12 columns

11. Check for missing values in each column.

```
df.isnull().sum()
```

✓ 0.0s

```
StudentID                  40
Name                       34
Gender                     48
AttendanceRate             40
StudyHoursPerWeek          50
PreviousGrade              33
ExtracurricularActivities  43
ParentalSupport            22
FinalGrade                 40
Study Hours                24
Attendance (%)             41
Online Classes Taken       25
dtype: int64
```

12. Drop rows with missing values.

```
df.dropna()
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
0	1.0	John	Male	85.0	15.0	78.0	1.0	High	80.0	4.8	59.0	False
1	2.0	Sarah	Female	90.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True
2	3.0	Alex	Male	78.0	10.0	65.0	0.0	Low	68.0	4.6	92.0	False
3	4.0	Michael	Male	92.0	25.0	90.0	3.0	High	92.0	2.9	96.0	False
6	7.0	Daniel	Male	70.0	8.0	60.0	0.0	Low	62.0	4.5	96.0	False
...	...	...	...	...	...	...	...	...	...	...	...	...
989	2116.0	Kimberly Pena	Female	91.0	30.0	88.0	2.0	High	68.0	3.6	79.0	False
991	7701.0	Anna Martinez	Male	85.0	30.0	70.0	3.0	Medium	90.0	0.4	76.0	False
993	3592.0	Monica Johnson	Female	90.0	25.0	60.0	1.0	Low	87.0	1.7	79.0	False
994	2787.0	Shannon Porter	Male	78.0	20.0	60.0	0.0	High	62.0	1.6	70.0	False
998	7636.0	Joseph Sherman	Male	88.0	17.0	60.0	2.0	High	85.0	0.9	53.0	True

645 rows × 12 columns

13. Fill missing values with the mean of a column.

```
df2 = df["StudyHoursPerWeek"].fillna(df["StudyHoursPerWeek"].mean())
df2
```

```
0      15.000000
1      20.000000
2      10.000000
3      25.000000
4      18.000000
         ...
995    20.000000
996    17.630526
997     8.000000
998    17.000000
999    10.000000
Name: StudyHoursPerWeek, Length: 1000, dtype: float64
```

14. Rename a column in a DataFrame.

```
df.rename(columns={"PreviousGrade": "Previous_Grade"}, inplace=True)
df
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
0	1.0	John	Male	85.0	15.0	78.0	1.0	High	80.0	4.8	59.0	False
1	2.0	Sarah	Female	90.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True
2	3.0	Alex	Male	78.0	10.0	65.0	0.0	Low	68.0	4.6	92.0	False
3	4.0	Michael	Male	92.0	25.0	90.0	3.0	High	92.0	2.9	96.0	False
4	5.0	Emma	Female	NaN	18.0	82.0	2.0	Medium	85.0	4.1	97.0	True
...	...	...	...	...	...	...	...	...	...	...	...	...
995	NaN	Kenneth Murray	Male	85.0	20.0	NaN	1.0	High	72.0	0.8	80.0	True
996	4497.0	Amy Stout	Female	91.0	NaN	86.0	0.0	High	90.0	3.9	80.0	True
997	1886.0	NaN	Male	85.0	8.0	82.0	2.0	Low	68.0	0.4	54.0	False
998	7636.0	Joseph Sherman	Male	88.0	17.0	60.0	2.0	High	85.0	0.9	53.0	True
999	8021.0	Maria Walls	Female	88.0	10.0	90.0	1.0	Medium	NaN	2.4	94.0	True

1000 rows × 12 columns

15. Change the data type of a column.

```
df["AttendanceRate"] = df["AttendanceRate"].astype(int)
df
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
0	1.0	John	Male	85	15.0	78.0	1.0	High	80.0	4.8	59.0	False
1	2.0	Sarah	Female	90	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True
2	3.0	Alex	Male	78	10.0	65.0	0.0	Low	68.0	4.6	92.0	False
3	4.0	Michael	Male	92	25.0	90.0	3.0	High	92.0	2.9	96.0	False
5	6.0	Olivia	Female	95	30.0	88.0	1.0	High	NaN	2.8	97.0	False
...	...	...	...	...	...	...	...	...	...	...	...	...
995	NaN	Kenneth Murray	Male	85	20.0	NaN	1.0	High	72.0	0.8	80.0	True
996	4497.0	Amy Stout	Female	91	NaN	86.0	0.0	High	90.0	3.9	80.0	True
997	1886.0	NaN	Male	85	8.0	82.0	2.0	Low	68.0	0.4	54.0	False
998	7636.0	Joseph Sherman	Male	88	17.0	60.0	2.0	High	85.0	0.9	53.0	True
999	8021.0	Maria Walls	Female	88	10.0	90.0	1.0	Medium	NaN	2.4	94.0	True

960 rows × 12 columns

16. Sort a DataFrame by a specific column.

```
df.sort_values("AttendanceRate", ascending=True)
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken
366	4659.0	Jill Kelly	Female	70	30.0	NaN	2.0	Low	85.0	-5.0	200.0	False
404	1769.0	Christopher Poole	Female	70	8.0	88.0	2.0	High	80.0	3.7	62.0	False
584	9220.0	Penny Hopkins	Female	70	12.0	86.0	1.0	High	80.0	2.8	95.0	False
552	4396.0	Michelle Ryan	Female	70	15.0	88.0	1.0	Low	92.0	0.1	62.0	False
901	2670.0	Bernard Whitaker	Male	70	8.0	65.0	1.0	High	90.0	4.9	61.0	True
...	...	...	...	...	...	...	...	...	...	...	...	...
137	5186.0	Larry Brown	Female	95	20.0	78.0	1.0	Low	62.0	0.9	74.0	True
663	3243.0	Thomas Mcgrath	Female	95	17.0	60.0	1.0	High	87.0	1.1	82.0	True
662	9908.0	Jodi Taylor	Female	95	18.0	82.0	1.0	High	72.0	2.9	59.0	True
854	1245.0	Terry Rodriguez	Female	95	15.0	60.0	3.0	Medium	92.0	0.3	67.0	False
235	6053.0	Mr. Martin Dominguez MD	Female	95	12.0	77.0	1.0	High	78.0	3.8	70.0	True

17. Find unique values in a column.

```
df["Name"].unique()
```

✓ 0.0s

```
<ArrowStringArray>
[            'John',            'Sarah',             'Alex',
          'Michael',           'Olivia',           'Daniel',
            'James',         'Isabella', 'Shannon Harrison',
     'Peter Davila',
 ...
    'Kimberly Pena',  'Rebecca Burnett',    'Anna Martinez',
     'James Garcia',   'Monica Johnson',   'Shannon Porter',
   'Kenneth Murray',        'Amy Stout',   'Joseph Sherman',
      'Maria Walls']
Length: 925, dtype: str
```

18. Count value occurrences in a column.

```
df["StudyHoursPerWeek"].value_counts()
```

✓ 0.0s

```
StudyHoursPerWeek
20.0    113
17.0    102
22.0     94
18.0     94
8.0      93
12.0     92
15.0     85
25.0     85
30.0     82
10.0     74
Name: count, dtype: int64
```

19. Group data by a column and calculate the mean.

```
df.groupby("AttendanceRate")["Previous_Grade"].mean()
```

✓ 0.0s

```
AttendanceRate
70    77.211538
78    76.346154
82    78.000000
85    77.296512
88    78.931818
90    77.161616
91    78.424528
92    77.530864
95    78.000000
Name: Previous_Grade, dtype: float64
```

20. Merge two DataFrames on a common column.

```
df1 = pd.DataFrame({"ID":[1,2], "Name":["A","B"]})
df2 = pd.DataFrame({"ID":[1,2], "Salary":[500,600]})

merged = pd.merge(df1, df2, on="ID")
merged
```

0.0s

	ID	Name	Salary
0	1	A	500
1	2	B	600

21. Create a pivot table from a DataFrame.

```
data = {
    "Name": ["A", "B", "C", "D"],
    "Department": ["IT", "HR", "IT", "HR"],
    "Salary": [50000, 40000, 60000, 45000]
}
df = pd.DataFrame(data)

pivot = pd.pivot_table(df,
                       values="Salary",
                       index="Department",
                       aggfunc="mean")
print(pivot)
```

0.0s

```
            Salary
Department
HR         42500.0
IT         55000.0
```

22. Apply a custom function to a column.

```
def fun(x):
    return x * 0.1

print(df["AttendanceRate"])

df["AttendanceRate"] = df["AttendanceRate"].apply(fun
df["AttendanceRate"]
```

✓ 0.0s

```
0      85.0
1      90.0
2      78.0
3      92.0
5      95.0
       ...
995    85.0
996    91.0
997    85.0
998    88.0
999    88.0
Name: AttendanceRate, Length: 960, dtype: float64

0      8.5
1      9.0
2      7.8
3      9.2
5      9.5
      ...
```

23. Use apply() with a lambda function.

```
df["AttendanceRate"].apply(lambda x: x * 10)
```

✓ 0.0s

```
0      85.0
1      90.0
2      78.0
3      92.0
5      95.0
       ...
995    85.0
996    91.0
997    85.0
998    88.0
999    88.0
Name: AttendanceRate, Length: 960, dtype: float64
```

24. Create a new column based on conditions.

```
df["Final_Grade"] = df["FinalGrade"].apply(lambda x: "High" if x > 85 else "Low")
df
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online Classes Taken	Final_Grade
0	1.0	John	Male	8.5	15.0	78.0	1.0	High	80.0	4.8	59.0	False	Low
1	2.0	Sarah	Female	9.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	True	High
2	3.0	Alex	Male	7.8	10.0	65.0	0.0	Low	68.0	4.6	92.0	False	Low
3	4.0	Michael	Male	9.2	25.0	90.0	3.0	High	92.0	2.9	96.0	False	High
5	6.0	Olivia	Female	9.5	30.0	88.0	1.0	High	NaN	2.8	97.0	False	Low
...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	NaN	Kenneth Murray	Male	8.5	20.0	NaN	1.0	High	72.0	0.8	80.0	True	Low
996	4497.0	Amy Stout	Female	9.1	NaN	86.0	0.0	High	90.0	3.9	80.0	True	High
997	1886.0	NaN	Male	8.5	8.0	82.0	2.0	Low	68.0	0.4	54.0	False	Low
998	7636.0	Joseph Sherman	Male	8.8	17.0	60.0	2.0	High	85.0	0.9	53.0	True	Low
999	8021.0	Maria Walls	Female	8.8	10.0	90.0	1.0	Medium	NaN	2.4	94.0	True	Low

960 rows × 13 columns

25. Perform multi-level indexing (hierarchical indexing).

```
df.set_index(["StudentID", "Name"])
df
```

✓ 0.0s Python

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	Previous_Grade	ExtracurricularActivities	ParentalSupport	FinalGrade	Study Hours	Attendance (%)	Online
0	1.0	John	Male	8.5	15.0	78.0	1.0	High	80.0	4.8	59.0	
1	2.0	Sarah	Female	9.0	20.0	85.0	2.0	Medium	87.0	2.2	70.0	
2	3.0	Alex	Male	7.8	10.0	65.0	0.0	Low	68.0	4.6	92.0	
3	4.0	Michael	Male	9.2	25.0	90.0	3.0	High	92.0	2.9	96.0	
5	6.0	Olivia	Female	9.5	30.0	88.0	1.0	High	NaN	2.8	97.0	
...	...	...	...	...	...	...	...	...	...	...	...	...
994	2787.0	Shannon Porter	Male	7.8	20.0	60.0	0.0	High	62.0	1.6	70.0	
996	4497.0	Amy Stout	Female	9.1	NaN	86.0	0.0	High	90.0	3.9	80.0	
997	1886.0	NaN	Male	8.5	8.0	82.0	2.0	Low	68.0	0.4	54.0	
998	7636.0	Joseph Sherman	Male	8.8	17.0	60.0	2.0	High	85.0	0.9	53.0	
999	8021.0	Maria Walls	Female	8.8	10.0	90.0	1.0	Medium	NaN	2.4	94.0	

921 rows × 13 columns

26. Resample time-series data by month.

```
df = pd.DataFrame(data)

df["Date"] = pd.to_datetime(df["Date"])
df.set_index("Date", inplace=True)

df.resample("ME").mean()
```

73] ✓ 0.0s

Date	Sales	Profit
2024-01-31	134.0	27.6
2024-02-29	224.0	55.0
2024-03-31	308.0	73.0
2024-04-30	422.0	95.0

27. Perform rolling window calculations.

```
    df["Rolling_Avg"] = df["Sales"].rolling(window=3).mean()
    df["Rolling_Avg"]
```

75] ✓ 0.0s

```
0           NaN
1           NaN
2    123.333333
3    133.333333
4    150.000000
5    166.666667
6    196.666667
7    210.000000
8    226.666667
9    233.333333
10   263.333333
11   273.333333
12   300.000000
13   303.333333
14   320.000000
15   346.666667
16   383.333333
17   410.000000
18   426.666667
19   430.000000
Name: Rolling_Avg, dtype: float64
```

28. Use loc and iloc for advanced indexing.

```
print(df.loc[0, "Name"])     # label-based
df.iloc[0, 1]          # position-based
```

✓ 0.0s

```
John

'John'
```

29. Detect and remove duplicate rows.

```
print(df[df.duplicated()])
df.drop_duplicates()
```

✓ 0.0s

```
   StudentID   Name  Marks
4        102  Sarah     90
7        103   Alex     78
```

	StudentID	Name	Marks
0	101	John	85
1	102	Sarah	90
2	103	Alex	78
3	104	Michael	88
5	105	Olivia	92
6	106	Emma	75

30. Perform a left join, right join, and inner join between two DataFrames.

```
df1.merge(df2, on="ID", how="inn
```

✓ 0.0s

	ID	Name	Salary
0	2	B	50000
1	3	C	60000

```
df1.merge(df2, on="ID", how="left")
```

✓ 0.0s

	ID	Name	Salary
0	1	A	NaN
1	2	B	50000.0
2	3	C	60000.0

```
df1.merge(df2, on="ID", how="right")
```

✓ 0.0s

	ID	Name	Salary
0	2	B	50000
1	3	C	60000
2	4	NaN	70000